

SLCO4A - Aula Prática 2

Sumário

Plot com o MATLAB.....	1
Plotando em duas dimensões.....	1
Comando linspace e logspace.....	4
Plotando várias funções em uma figura.....	4
Configurando opções do plot.....	5
Comando hold.....	6
Formatando uma figura.....	7
Plotando em diferentes figuras.....	8
Funções por partes.....	9
Outros comandos para plotar uma função.....	10
Plot de funções de tempo discreto.....	14
Gráfico em coordenadas polares.....	15
Gráfico tridimensional.....	16

Plot com o MATLAB

Plotando em duas dimensões

a) Plot a função $y(x) = x^2$, $-2 \leq x \leq 2$

```
clear all; close all
clc
x0=-2:2
```

```
x0 = 1x5
    -2    -1     0     1     2
```

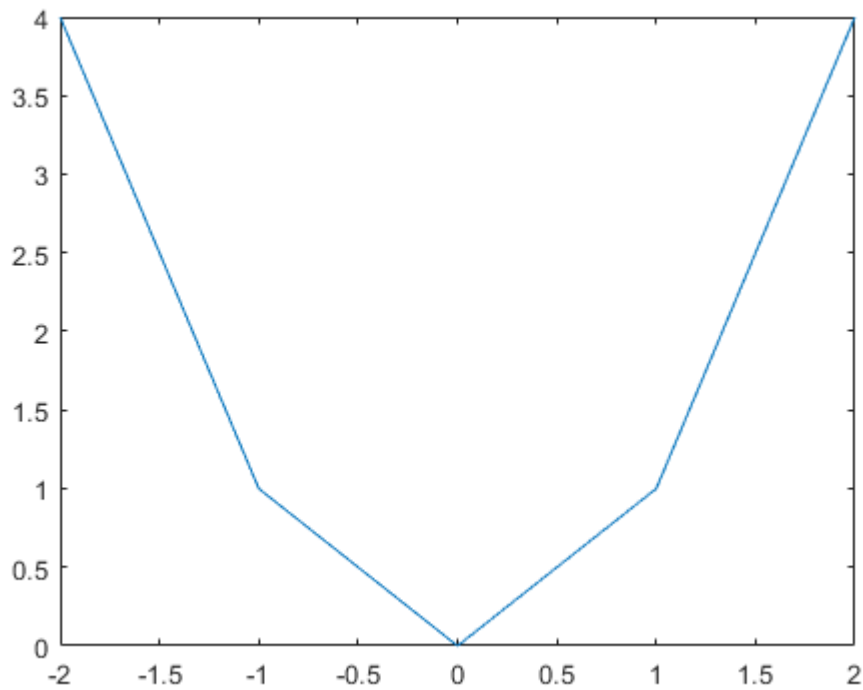
```
length(x0)
```

```
ans = 5
```

```
y0=x0.^2;
length(y0)
```

```
ans = 5
```

```
plot(x0,y0)
```



Observação 1: Note que a representação gráfica correspondeu a uma quantidade de cinco pares ordenados (x,y) , que resultou em um gráfico distorcido da representação desejada para a função. Neste tipo de caso deve ser aumentar o número de pontos da variável independente para obter uma melhor representação gráfica.

b) Aumente o número de ponto da variável independente x , considerando passo de incremento de 0.1 e refaça o item anterior

```
x=-2:0.1:2
```

```
x = 1×41
-2.0000 -1.9000 -1.8000 -1.7000 -1.6000 -1.5000 -1.4000 -1.3000 ...
```

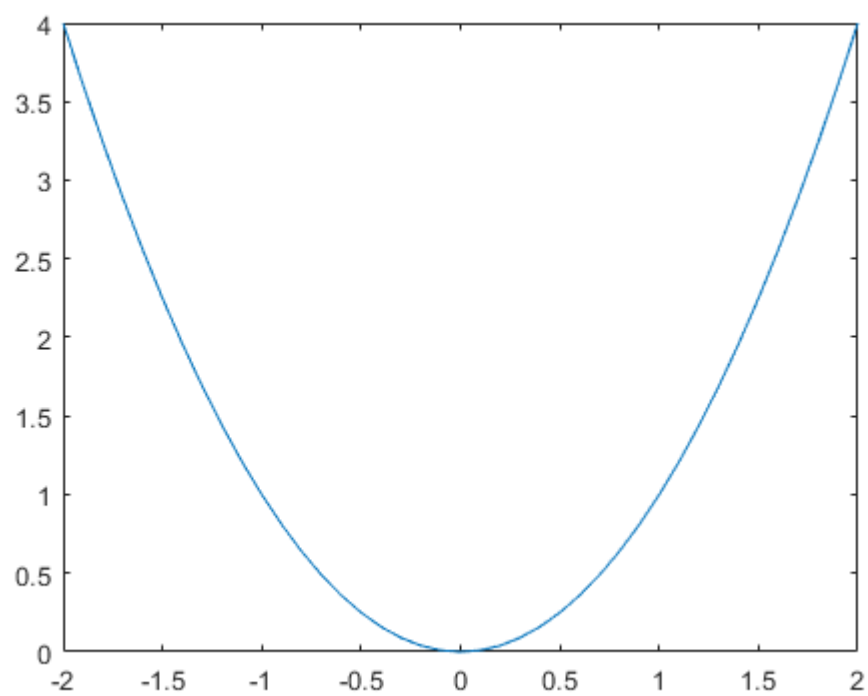
```
y=x.^2
```

```
y = 1×41
4.0000 3.6100 3.2400 2.8900 2.5600 2.2500 1.9600 1.6900 ...
```

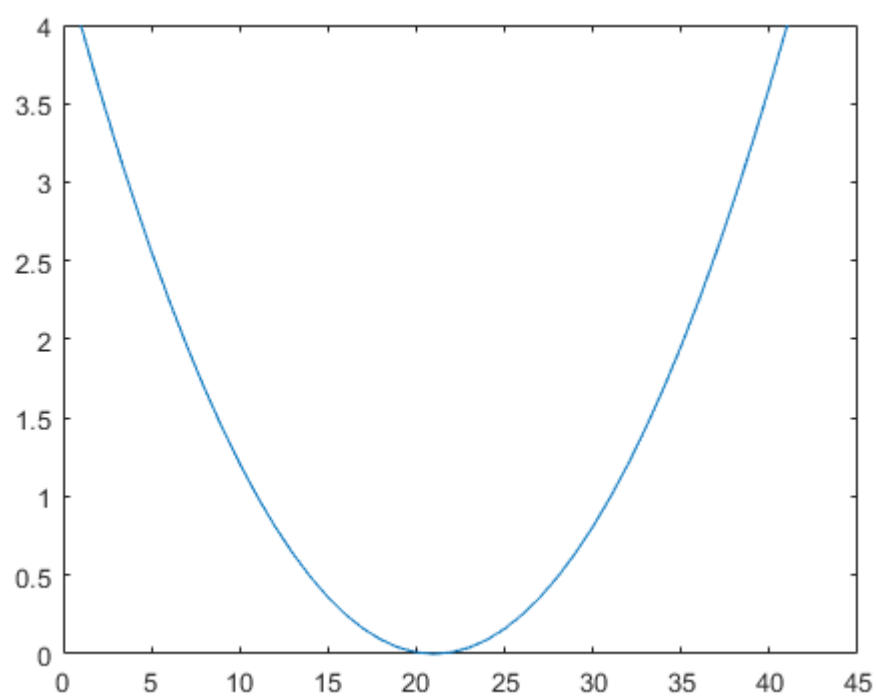
```
length(y)
```

```
ans = 41
```

```
plot(x,y)
```



```
plot(y)
```



```
y=x.^2;  
length(y)
```

```
ans = 41
```

Observação 2: Há algumas opções disponíveis no canto superior direito das figuras. As opções permitem exportar, visualizar valor numérico, dar zoom e editar em janela gráfica externa.

Comando linspace e logspace

```
x=0:0.25:1
```

```
x = 1×5  
    0    0.2500    0.5000    0.7500    1.0000
```

```
x=linspace(0,1,5)
```

```
x = 1×5  
    0    0.2500    0.5000    0.7500    1.0000
```

```
%Criar um vetor com cinco pontos espaçados logaritmicamente entre 10^0 até 10^1  
logspace(0,1,5)
```

```
ans = 1×5  
    1.0000    1.7783    3.1623    5.6234   10.0000
```

Plotando várias funções em uma figura

c) Plot as seguintes funções em uma mesma figura. $y(x) = x^2 \cos(x)$, $g(x) = x \cos(x)$ e $f(x) = 2^x \sin(x)$, $0 \leq x \leq 2\pi$, considerando 100 pontos.

```
x=linspace(0,2*pi,100)
```

```
x = 1×100  
    0    0.0635    0.1269    0.1904    0.2539    0.3173    0.3808    0.4443 ...
```

```
y=(x.^2).*cos(x)
```

```
y = 1×100  
    0    0.0040    0.0160    0.0356    0.0624    0.0957    0.1346    0.1782 ...
```

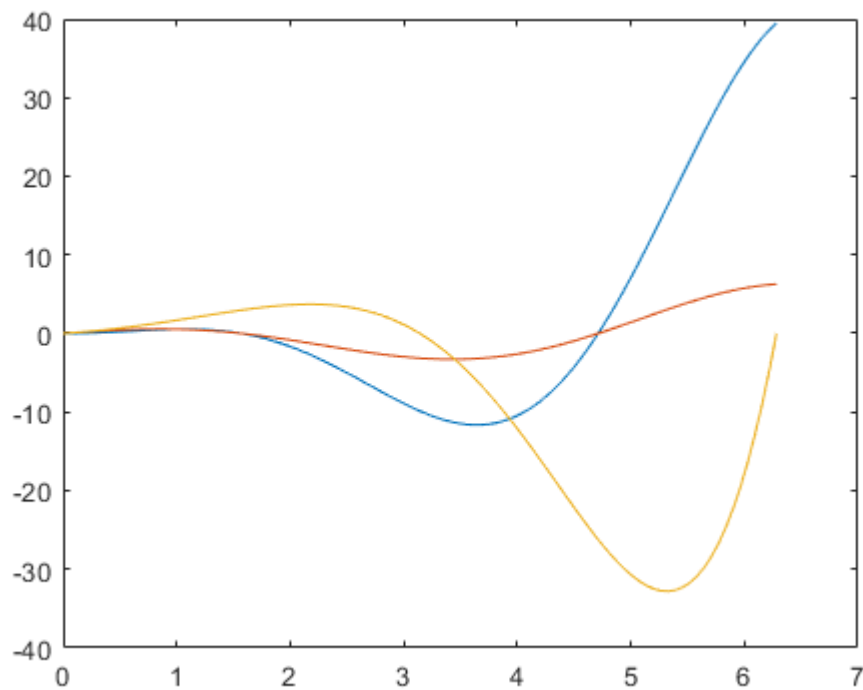
```
g=x.*cos(x)
```

```
g = 1×100  
    0    0.0633    0.1259    0.1870    0.2457    0.3015    0.3535    0.4011 ...
```

```
f=(2.^x).*sin(x)
```

```
f = 1×100  
    0    0.0663    0.1382    0.2160    0.2995    0.3888    0.4839    0.5848 ...
```

```
plot(x,y,x,g,x,f)
```



Observação 3: Note que a figura foi criada com cores predefinidas e linha do tipo sólido. É possível criar gráficos usando cores, símbolos e tipos de linhas personalizados por meio da inserção de outros argumentos na função `plot`, vide opções por meio do comando `help plot`.

Configurando opções do plot

d) Plot as funções $y_1(x) = \sin(x)$ e $y_2(x) = \cos(x)$, $-1 \leq x \leq 1$ com 15 elementos. Para a primeira função utilize cor preta, linha pontilhada e marcadores do tipo pentagrama. Enquanto que para segunda função utilize cor vermelha, linha tracejada e marcadores do tipo círculo.

```
x=linspace(-1,1,15)
```

```
x = 1×15
-1.0000 -0.8571 -0.7143 -0.5714 -0.4286 -0.2857 -0.1429 0 ...
```

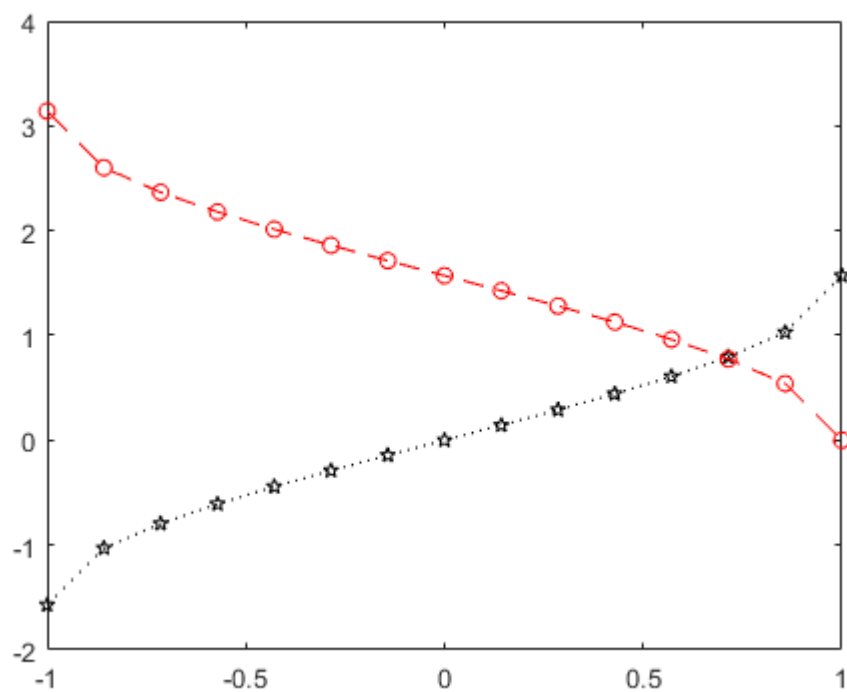
```
y1=asin(x)
```

```
y1 = 1×15
-1.5708 -1.0297 -0.7956 -0.6082 -0.4429 -0.2898 -0.1433 0 ...
```

```
y2=acos(x)
```

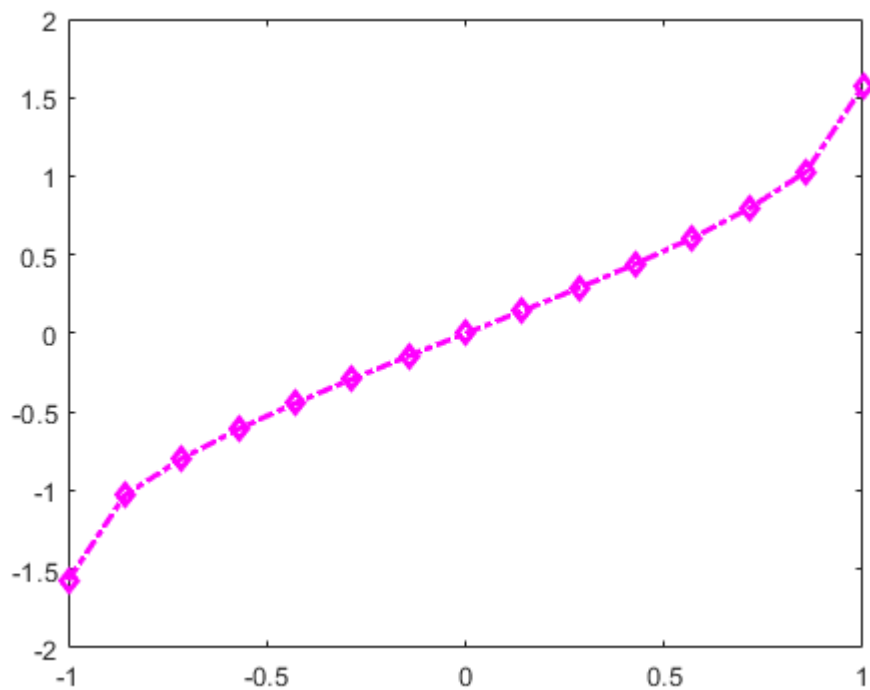
```
y2 = 1×15
3.1416 2.6005 2.3664 2.1790 2.0137 1.8605 1.7141 1.5708 ...
```

```
plot(x,y1,'k:p',x,y2,'r--o')
```



Outra forma de sintaxe para o comando `plot` que especifica cor e tipo de linha é da seguinte forma

```
plot(x,y1, 'Color',[1,0,1], 'LineWidth',2, "Marker", "diamond", "Linestyle", "-.")
```



Comando `hold`

Ao repetir a execução do comando `plot` para uma figura prévia, a representação gráfica obtida anteriormente é descartada. Para manter o gráfico anterior e acrescentar um novo gráfico por meio da repetição do comando `plot` deve-se utilizar o comando `hold on`. Veja o exemplo

```
x=linspace(0,2*pi,150)
```

```
x = 1×150  
      0      0.0422      0.0843      0.1265      0.1687      0.2108      0.2530      0.2952 ...
```

```
plot(x,cos(x))  
hold on  
plot(x,sin(x))
```

Formatando uma figura

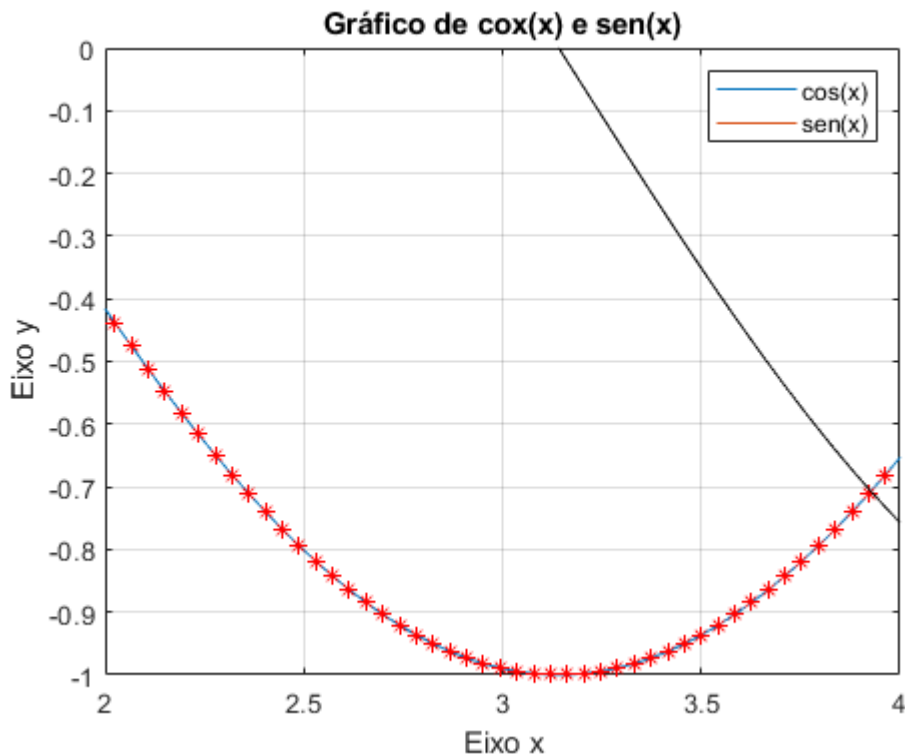
Há alguns comandos úteis para formatação de figuras, tais como:

`grid`-`title`-`xlabel`-`ylabel`-`legend`-`text`-`gtext`-`axis`-`xlim`-`ylim`

```
x=linspace(0,2*pi,150)
```

```
x = 1×150  
      0      0.0422      0.0843      0.1265      0.1687      0.2108      0.2530      0.2952 ...
```

```
plot(x,cos(x), 'r*',x,sin(x), 'k')  
legend('cos(x)', 'sen(x)')  
grid  
xlabel('Eixo x')  
ylabel('Eixo y')  
title('Gráfico de cos(x) e sen(x)')  
text(3,0.3, 'sen(x)')  
text(2,0.3, 'cos(x)')  
axis([2,4, -1,0])
```



Plotando em diferentes figuras

Para criar diferentes figuras pode-se especificar a partir do comando `figure(1) ... figure(k)` e utilizar para cada figura o comando `plot`.

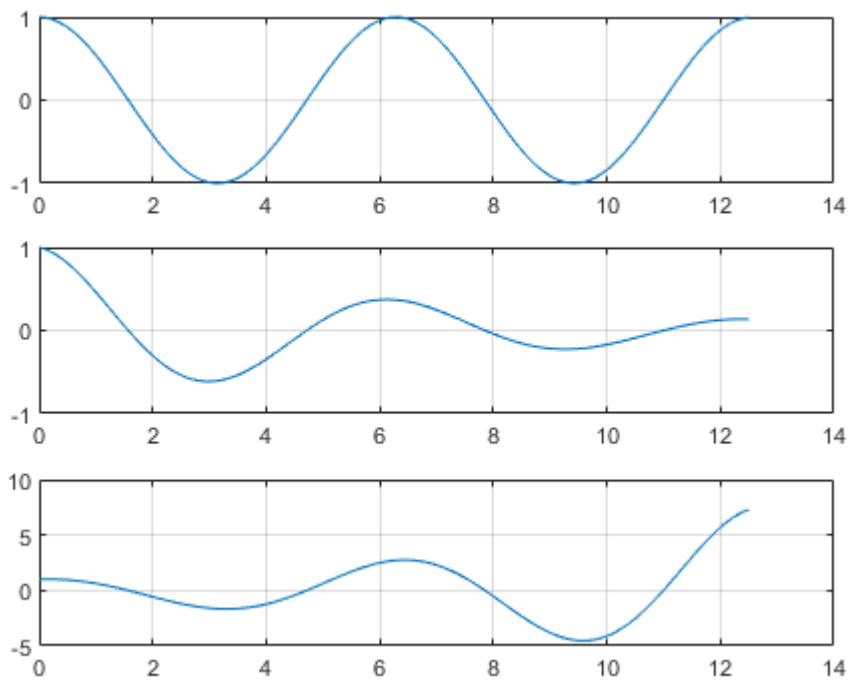
Mas também é possível criar múltiplas figuras dentro de uma única figura por meio do comando `subplot(m,n,p)`. Com este comando cria-se dentro de uma mesma janela de figura que estará dividida em $m \times n$ pequenas subfiguras.

e) Utilizando o comando `subplot` plot as seguintes funções $f(x) = \cos(x)$, $g(x) = e^{-\frac{x}{2\pi}}\cos(x)$, $f(x) = e^{\frac{x}{2\pi}}\cos(x)$ em uma única figura contendo 3 subfiguras disposta em um arranjo 3x1

```
figure
x=0:0.1:4*pi
```

```
x = 1x126
      0      0.1000      0.2000      0.3000      0.4000      0.5000      0.6000      0.7000 ...
```

```
subplot(3,1,1)
plot(x,cos(x))
grid on
subplot(3,1,2)
plot(x,exp(-x/(2*pi)).*cos(x))
grid on
subplot(3,1,3)
plot(x,exp(x/(2*pi)).*cos(x))
grid on
```

Funções por partes

f) Obtenha a representação gráfica de uma função definida por partes, tal que

$$f(t) = \begin{cases} 1, & -2 \leq t \leq 2 \\ 0, & 2 < t < 5 \\ t \sin(4\pi t) & 5 \leq t \leq 8 \end{cases}$$

```
t1=-2:0.01:2
```

```
t1 = 1×401
    -2.0000    -1.9900    -1.9800    -1.9700    -1.9600    -1.9500    -1.9400    -1.9300 ...
```

```
t2=2.01:0.01:4.99
```

```
t2 = 1×299
    2.0100    2.0200    2.0300    2.0400    2.0500    2.0600    2.0700    2.0800 ...
```

```
t3=5:0.01:8
```

```
t3 = 1×301
    5.0000    5.0100    5.0200    5.0300    5.0400    5.0500    5.0600    5.0700 ...
```

```
f1=ones(size(t1))
```

```
f1 = 1×401
    1     1     1     1     1     1     1     1     1     1     1     1     1 ...
```

```
f2=zeros(size(t2))
```

```
f2 = 1×299
    0     0     0     0     0     0     0     0     0     0     0     0     0 ...
```

```
f3=t3.*sin(4*pi*t3)
```

```
f3 = 1×301  
-0.0000    0.6279    1.2484    1.8517    2.4280    2.9683    3.4638    3.9065 ...
```

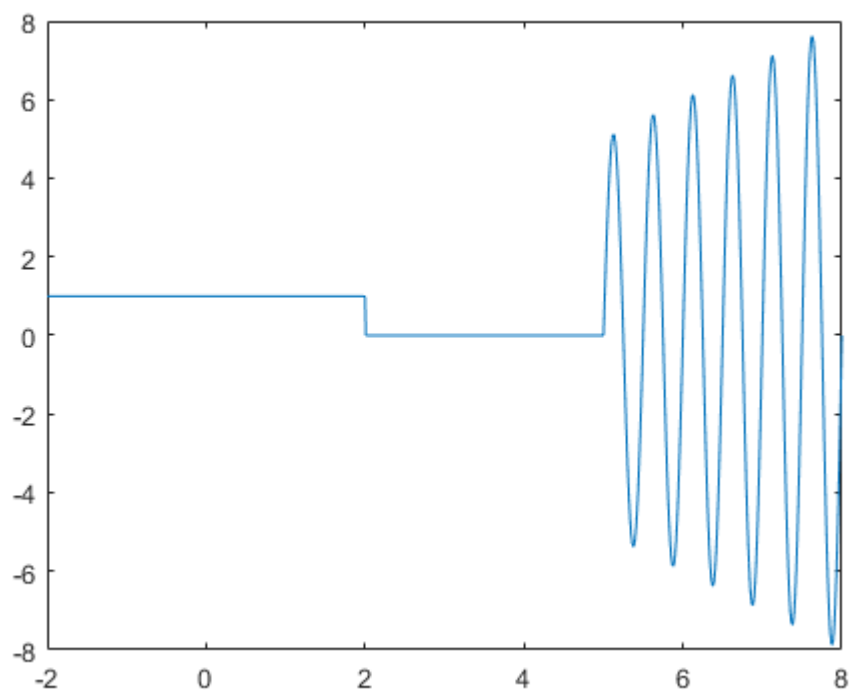
```
t=[t1 t2 t3]
```

```
t = 1×1001  
-2.0000   -1.9900   -1.9800   -1.9700   -1.9600   -1.9500   -1.9400   -1.9300 ...
```

```
f=[f1 f2 f3]
```

```
f = 1×1001  
1    1    1    1    1    1    1    1    1    1    1    1    1 ...
```

```
figure  
plot(t,f)
```



Outros comandos para plotar uma função

Além do comando `plot` há outros comandos que podem ser empregados para obter uma representação gráfica de uma função, tais como:

`loglog-semilogx-semilogy-area-fplot-ezplot`

f) Considere a função $y(x) = 2x + 30$, $1 \leq x \leq 1000$. Obtenha diferentes representações gráficas utilizando outros comandos além do `plot`.

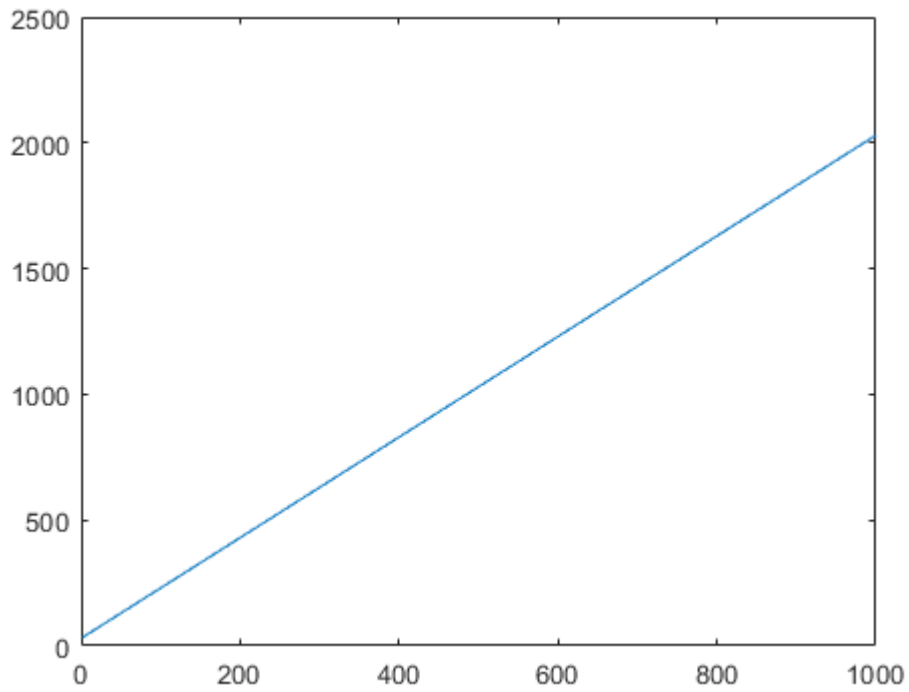
```
x=1:1000
```

```
x = 1×1000  
1    2    3    4    5    6    7    8    9   10   11   12   13 ...
```

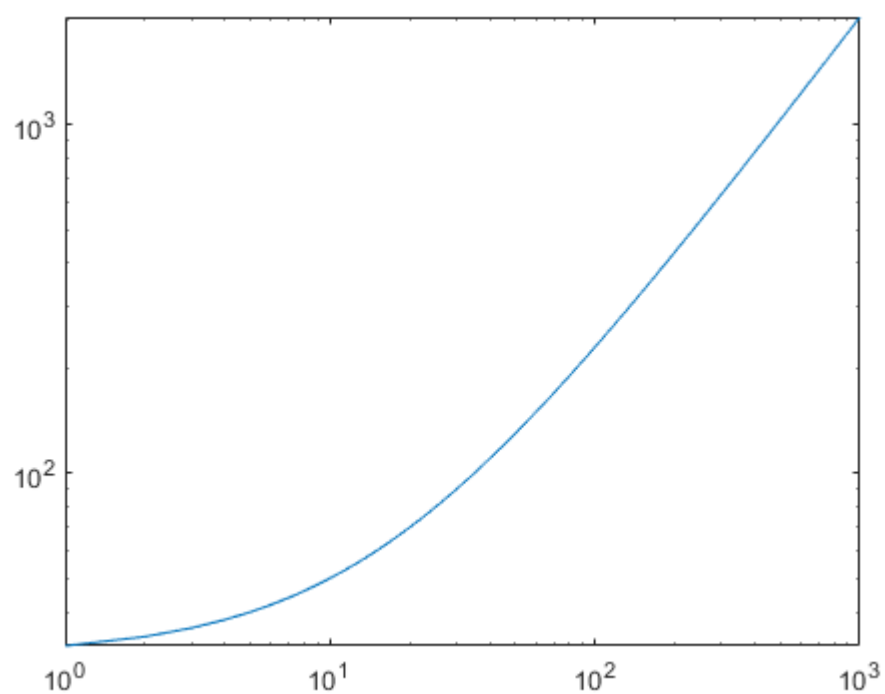
```
y=2*x+30
```

```
y = 1×1000  
32 34 36 38 40 42 44 46 48 50 52 54 56 ⋯
```

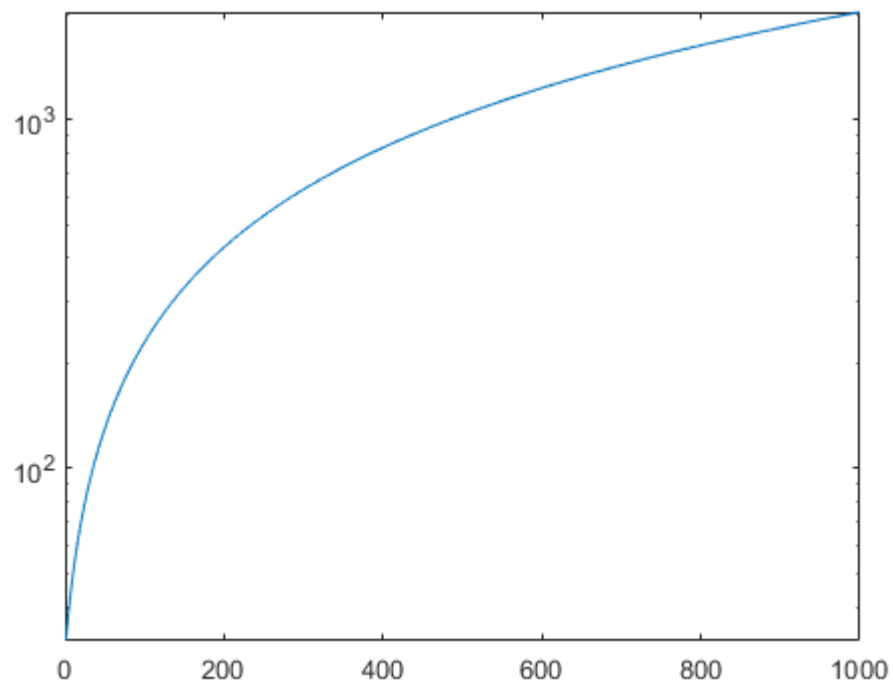
```
figure  
plot(x,y)
```



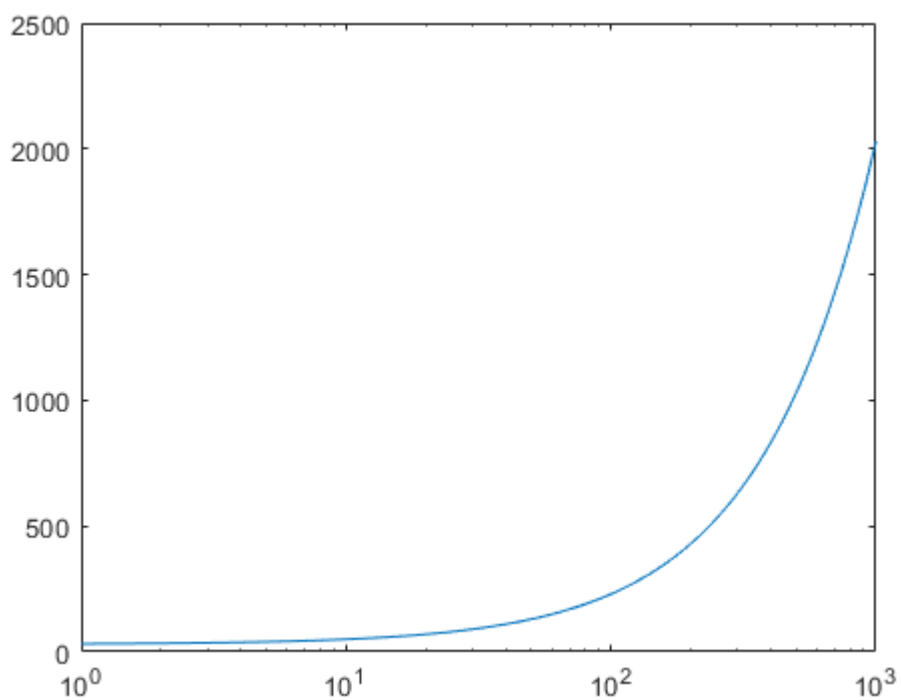
```
figure  
loglog(x,y)
```



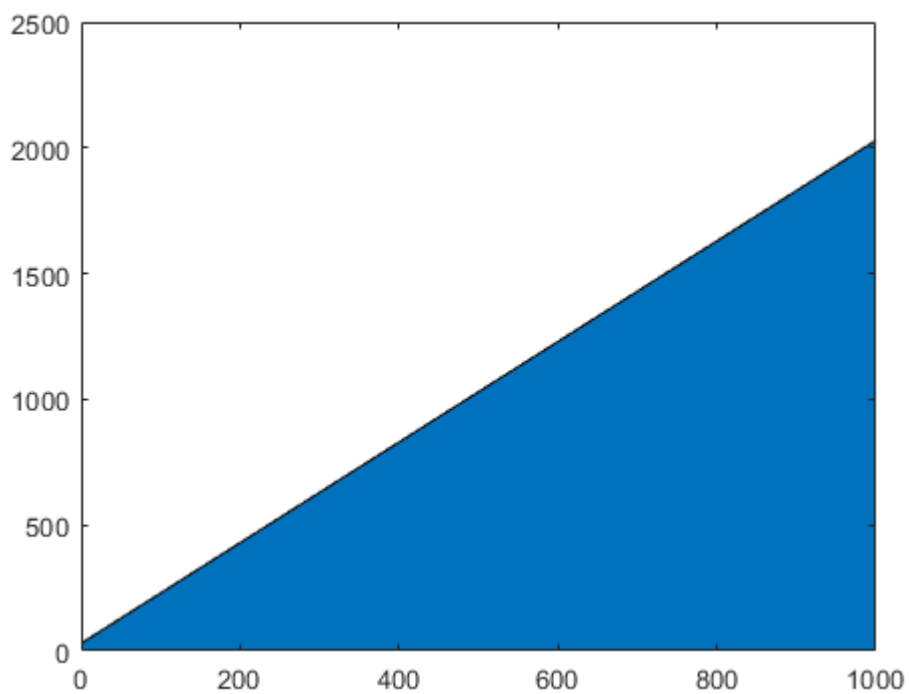
```
figure  
semilogy(x,y)
```



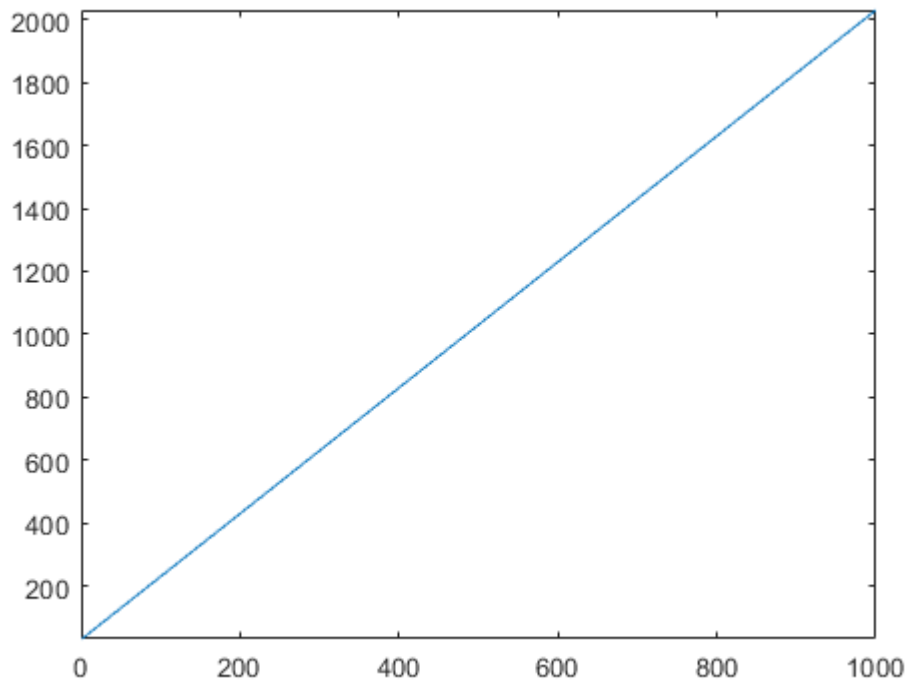
```
figure  
semilogx(x,y)
```



```
figure
area(x,y)
```



```
figure
fplot(@(x)2.*x+30,[1,1000])
```



```
%fplot('2*x+30',[1,1000])
%ezplot('2*x+30',[1,1000])
```

Plot de funções de tempo discreto

g) Considere a função de tempo discreto $f(k) = k^2$, $k \in \mathbb{Z} : -3 \leq k \leq 3$.

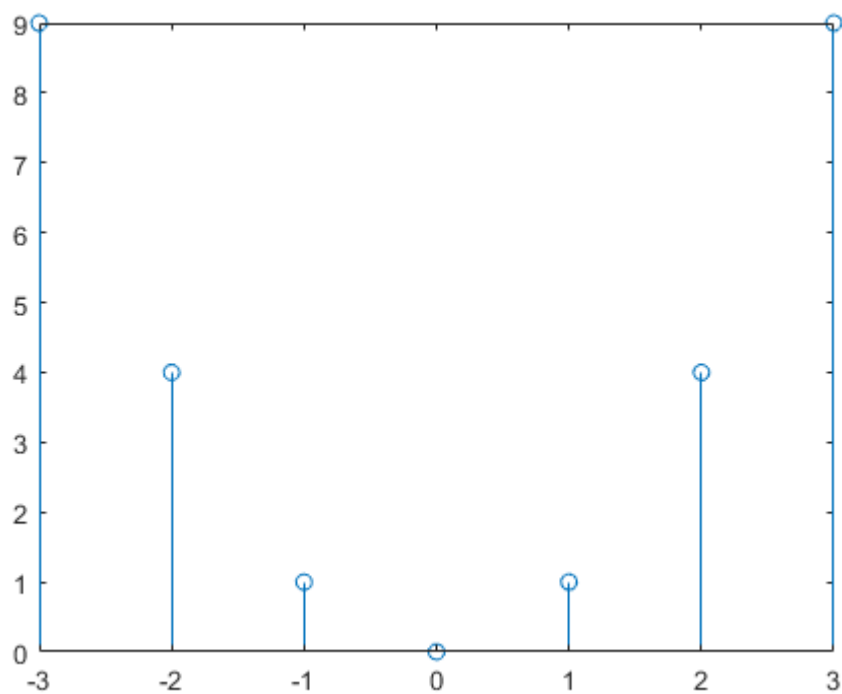
```
k=-3:3
```

```
k = 1×7
    -3    -2    -1     0     1     2     3
```

```
f=k.^2
```

```
f = 1×7
     9     4     1     0     1     4     9
```

```
stem(k,f)
```



```
stem(k,f,'r*')
```

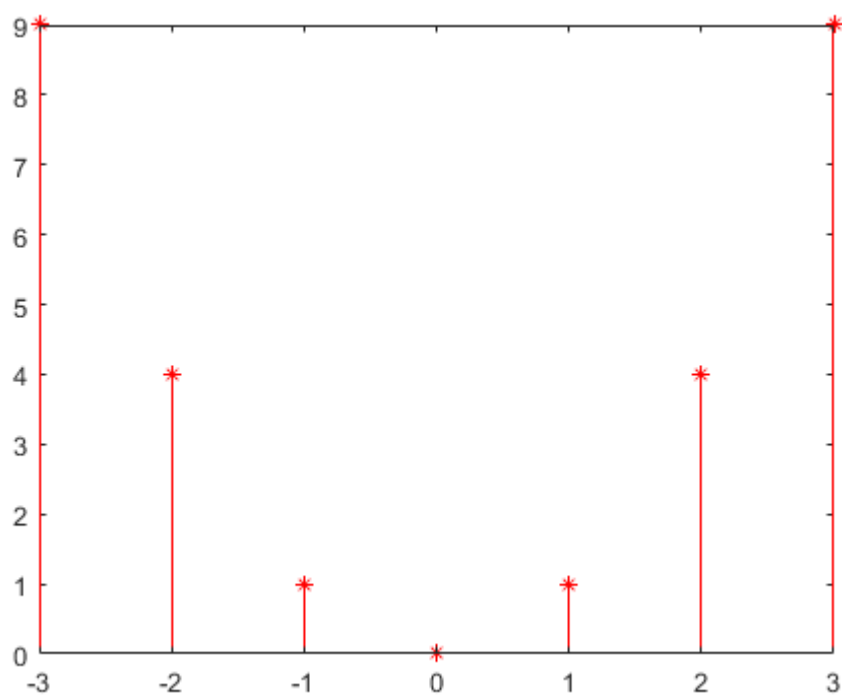


Gráfico em coordenadas polares

```
theta = linspace(0,4*pi,50)
```

```
theta = 1x50
```

0	0.2565	0.5129	0.7694	1.0258	1.2823	1.5387	1.7952 ...
---	--------	--------	--------	--------	--------	--------	------------

```
c=theta
```

```
c = 1×50
```

0	0.2565	0.5129	0.7694	1.0258	1.2823	1.5387	1.7952 ...
---	--------	--------	--------	--------	--------	--------	------------

```
polarplot(theta,c, '-*')
```

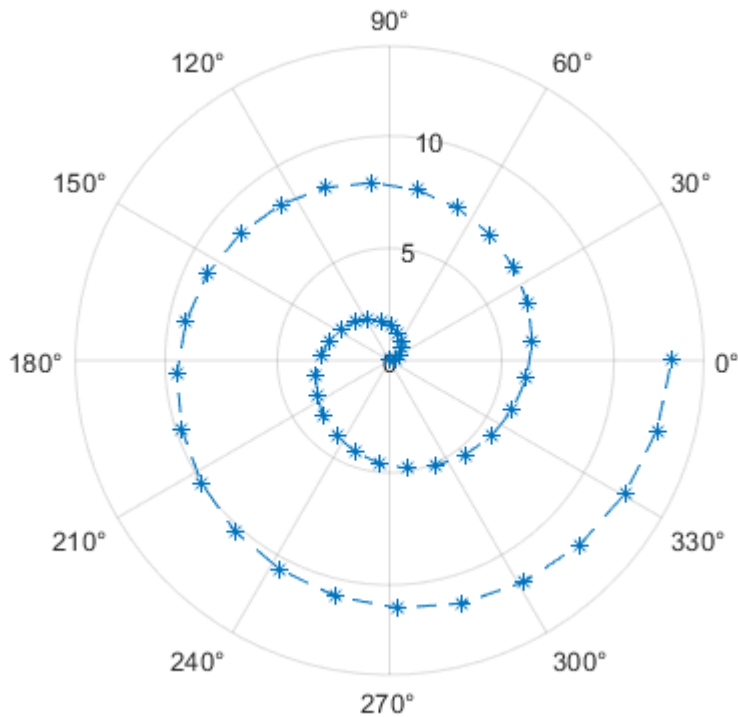


Gráfico tridimensional

```
x=0:.1:100
```

```
x = 1×1001
```

0	0.1000	0.2000	0.3000	0.4000	0.5000	0.6000	0.7000 ...
---	--------	--------	--------	--------	--------	--------	------------

```
y=cos(x)
```

```
y = 1×1001
```

1.0000	0.9950	0.9801	0.9553	0.9211	0.8776	0.8253	0.7648 ...
--------	--------	--------	--------	--------	--------	--------	------------

```
z=sin(x)
```

```
z = 1×1001
```

0	0.0998	0.1987	0.2955	0.3894	0.4794	0.5646	0.6442 ...
---	--------	--------	--------	--------	--------	--------	------------

```
plot3(x,y,z)
xlabel('x')
ylabel('y=cos(x)')
zlabel('z=sin(x)')
```